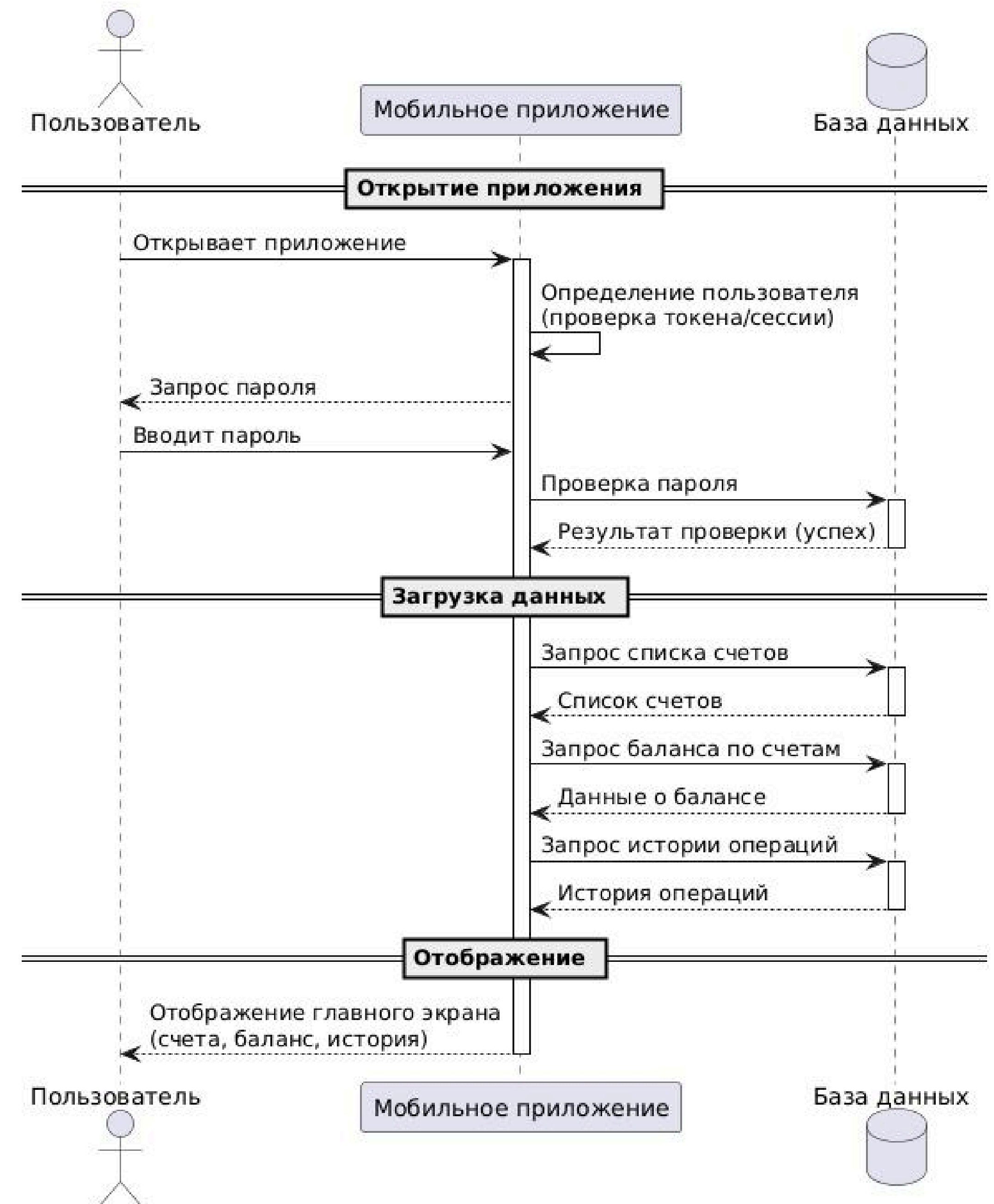


Основы проектирования баз данных

Данные — это новый вид сырья XXI века. Но сами по себе они бесполезны. Ценность появляется только тогда, когда данные правильно организованы, защищены и доступны для обработки.

Почему базы данных стали основой современного мира

Каждое современное приложение постоянно обращается к базе данных



Мир, построенный на данных

Банковская сфера

хранит информацию о:

-  Клиентах
-  Банковских счетах
-  Кредитах и вкладах
-  Денежных переводах
-  Курсах валют

 Количество записей — миллиарды

Интернет-магазины

система определяет:

- ✓ Существует ли товар
- ✓ Есть ли на складе
- ✓ Актуальная цена и скидка
- ✓ Отзывы покупателей

Без БД интернет-магазин становится бесполезным

Мир, построенный на данных

Социальные сети

система получает:

-  Сведения о пользователе
-  Фотографию профиля
-  Список друзей
-  Последние публикации
-  Комментарии и отметки «Нравится»
-  Уведомления

Государственные информационные системы

Хранится информация о:

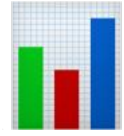
-  Гражданах
 -  Документах
 -  Налогах
 -  Недвижимости
 -  Транспортных средствах
 -  Медицинских полисах
-  Количество записей — **десятки миллиардов**

База данных — это невидимая часть любой информационной системы

Что видит пользователь	Что происходит внутри
Красивый интерфейс	Обращение к БД
Быстрая работа	Оптимизация запросов
Отсутствие ошибок	Контроль целостности
Сохранение данных	Транзакции

Без памяти человек теряет жизненный опыт.
Без БД система теряет смысл существования.

Компонент	Аналогия
Интерфейс	Органы чувств
Бизнес-логика	Мозг
База данных	Память

 **Крупнейшие компании ежедневно** обрабатывают петабайты данных

1 петабайт $\approx$ 1 000 000 гигабайт

Если записывать данные такого объёма на DVD-диски, их количество исчислялось бы сотнями тысяч

Данные, информация и знания

Почему важно различать эти понятия?

? Что такое данные?

Многие отвечают: «Это цифры», «Это любая информация», «Это одно и то же»

✗ На самом деле — это разные понятия

Для разработчика понимание этой разницы имеет **большое значение**

Если неправильно понимать природу данных, очень сложно спроектировать хорошую базу данных

Что такое данные

📖 Данные — от лат. *datum* — «данное», «то, что предоставлено»

Данные — это сырьё, которое ещё не обработано

19 21 18 22 20

? Что означают эти числа?

Это могут быть:

- Возраст студентов
- Температура воздуха
- Количество сотрудников
- Результаты экзамена

Без дополнительного контекста — это просто данные

Пример из реальной жизни:

Штрих-код товара:

Для человека — набор цифр

Для системы — ключ к информации о товаре



Что такое информация

 **Информация** — это данные, которые получили смысл после обработки или интерпретации

Информация = Данные + Контекст

19 21 18 22 20

Информация всегда отвечает на вопрос ?

 *Добавляем контекст:*

 **Возраст студентов группы ИСП-21**

Теперь мы можем сделать выводы:

-  Самый младший — 18 лет
-  Самый старший — 22 года
-  Средний возраст ≈ 20 лет

Данные
512

Информация
Количество студентов первого курса — 512 человек

Что такое знания

 **Знания** — это способность принимать решения на основе информации

Информация сообщает факты

Знания позволяют принимать решения

 Информация: студент получил оценки **5, 5, 4, 5, 5**

 Правило: «Если средний балл выше 4,75, студент получает повышенную стипендию»

 Решение: студент имеет право на повышенную стипендию

Данные → Информация → Знания

Сырьё → Смысл → Действие

Модель DIKW

Data → Information → Knowledge → Wisdom

Уровень	Описание	Пример
 Данные	Сырьё, факты	19, 21, 18
 Информация	Осмысленные данные	«Возраст студентов»
 Знания	Способность принимать решения	«Средний балл → стипендия»
 Мудрость	Принятие правильных решений	Оценка последствий

«Garbage In — Garbage Out»
«Мусор на входе — мусор на выходе»

Почему важна структура данных

Плохие данные нельзя исправить хорошей программой

✗ Пример ошибки: дата рождения студента указана как **2098 год**

Результат:

- Возраст — отрицательное число
- Отказ в регистрации
- Неправильные отчёты

Структурированные данные

Имя: Иван
Возраст: 19
Группа: ИСП-21
Средний балл: 4.8

- ✓ Каждая характеристика на своём месте
- ✓ Легко хранить в таблице

Неструктурированные данные

«Сегодня Иван пришёл на занятия раньше обычного. После второй пары он посетил библиотеку...»

- ✗ Нет заранее известных полей
- ✗ Сложнее обрабатывать

Полуструктурированные данные

 **JSON — самый известный пример**

```
{  
  "student": "Иванов Иван",  
  "group": "ИСП-21",  
  "age": 19,  
  "subjects": [  
    "Базы данных",  
    "Web-разработка",  
    "Алгоритмы"  
  ]  
}
```

Характеристики:

-  Присутствует определённая структура
-  Различные документы могут содержать разные поля
-  Широко используется в веб-приложениях

Вывод:

Структура данных определяет, какая модель базы данных окажется наиболее эффективной

Что такое база данных

Определение

База данных — это организованная совокупность взаимосвязанных данных, предназначенная для долговременного хранения, поиска, изменения и совместного использования.

Ключевые слова:

-  Организованная
-  Взаимосвязанная

Пример плохой организации

 Текстовый документ:

Иванов Иван, ИСП-21, 19 лет
Петров Пётр, ИСП-22, 20 лет
Сидоров Алексей, ИСП-21, 18 лет

- ? Как быстро найти студента?
- ? Как узнать количество студентов в группе?
- ? Как изменить одну запись?

Основные задачи базы данных

Задача	Описание
 Долговременное хранение	Сохранение после выключения компьютера
 Быстрый поиск	Нахождение записи за доли секунды
 Изменение информации	Обновление данных
 Добавление записей	Расширение объёма информации
 Удаление информации	Корректное удаление без нарушения системы
 Совместная работа	Одновременная работа множества пользователей
 Защита информации	Разграничение прав доступа

Что хранится в базе данных?

Тип данных	Пример
 Текст	Имена, описания
 Числа	Возраст, цены, количество
 Даты	Дата рождения, регистрации
 Время	Время операции
 Логические	Да/Нет, Истина/Ложь
 Изображения	Фотографии товаров
 Документы	PDF, Word
 Координаты	Геоданные
 JSON/XML	Полуструктурированные данные

⚠ Важное замечание

База данных — это НЕ программа

✗ Неправильно

✓ Правильно

«База данных показывает список товаров»

«Приложение показывает список товаров из БД»

«MySQL отображает интерфейс»

«СУБД MySQL управляет данными»

«База данных проверила пароль»

«Бизнес-логика проверила пароль по данным из БД»

База данных — организованное хранилище информации

СУБД — программное обеспечение для управления этим хранилищем

Почему обычные файлы не подходят для хранения данных

Проблема №1: Сложность поиска

Количество записей	Поиск в файле	Поиск в БД
50	✓ Возможно	✓ Мгновенно
500	⌚ Долго	✓ Мгновенно
5 000	✗ Очень долго	✓ Мгновенно

Пример сложного запроса:

- Все студенты группы ИСП-21
- Студенты старше 18 лет
- Средний балл выше 4.5



Проблема №2: Дублирование данных

✗ Один студент может встречаться в десятках документов

Иванов Иван:

-  Список студентов
-  Журнал успеваемости
-  Библиотечная система
-  Общежитие
-  Бухгалтерия

При смене фамилии нужно изменить **езде!**



Проблема №3: Несогласованность информации

✗ Студент переведён из ИСП-21 в ИСП-22

Информация изменена:

- **✓** В журнале успеваемости
- **✓** В расписании
- **✗** НЕ изменена в библиотеке

? Какая информация правильная?

Проблема №4: Сложность совместной работы

 Если файл открыт одним пользователем, остальные не могут его редактировать

Конфликты:

- Два пользователя одновременно изменяют одну запись
- Какое изменение сохранить?
- Как избежать потери данных?

Проблема №5: Отсутствие контроля корректности

- ✗ В поле «Возраст» введено -15
- ✗ В поле «Дата рождения» введено 31 февраля

Файл спокойно сохранится!

Проблема №6: Отсутствие разграничения доступа

✗ Если пользователь получил доступ к файлу, он может:

- Просматривать информацию
- Изменять её
- Удалять записи
- Копировать документ

Недопустимо для крупных систем!

Проблема №7: Отсутствие резервного копирования

- 💣 Файл удалён
 - 💣 Жёсткий диск вышел из строя
 - ❌ Информация потеряна **безвозвратно**
-

Итог

Файлы подходят только для **небольшого объёма** информации

По мере роста данных проявляются серьёзные недостатки

Сравнение: Файлы vs База данных

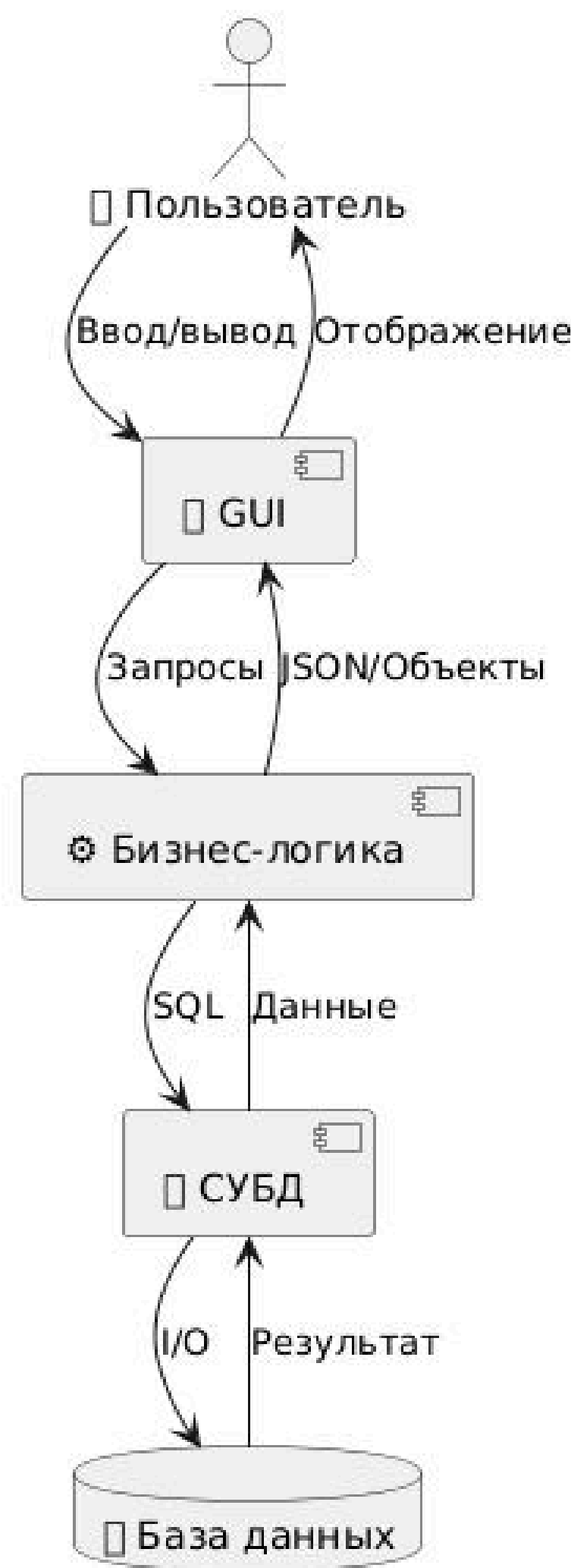
Характеристика	 Файлы	 База данных
Объём данных	Маленький	Миллионы/миллиарды записей
Поиск	Ограниченный	Быстрый по множеству критериев
Дублирование	Частое	Минимизировано
Согласованность	Высокий риск	Целостность данных
Совместная работа	Сложно	Множество пользователей
Контроль данных	Отсутствует	Проверка перед сохранением
Защита	Ограниченная	Гибкая система прав
Резервное копирование	Вручную	Автоматизированное

Вывод:

По мере роста данных преимущества БД становятся всё более очевидными

Место БД в современной информационной системе

Компонентная архитектура (упрощенная)



Роль каждого компонента

Компонент	Функция
 Пользователь	Использует систему
 Интерфейс	Отображение информации, получение команд
 Бизнес-логика	Принятие решений, правила программы
 СУБД	Управление данными, выполнение запросов
 База данных	Хранение информации

Что делает каждый компонент

◆ Пользовательский интерфейс

- Окна, кнопки, формы ввода
 - Меню, таблицы, изображения
 - Отображает данные, но не хранит их
-

◆ Бизнес-логика

Набор правил, определяющих поведение программы:

- ☒ Можно ли оформить заказ?
- ☒ Достаточно ли товара на складе?
- ☒ Имеет ли пользователь право изменить оценку?
- ☒ Хватает ли денег на счете?

Отвечает на вопрос: «Что должна сделать программа?»

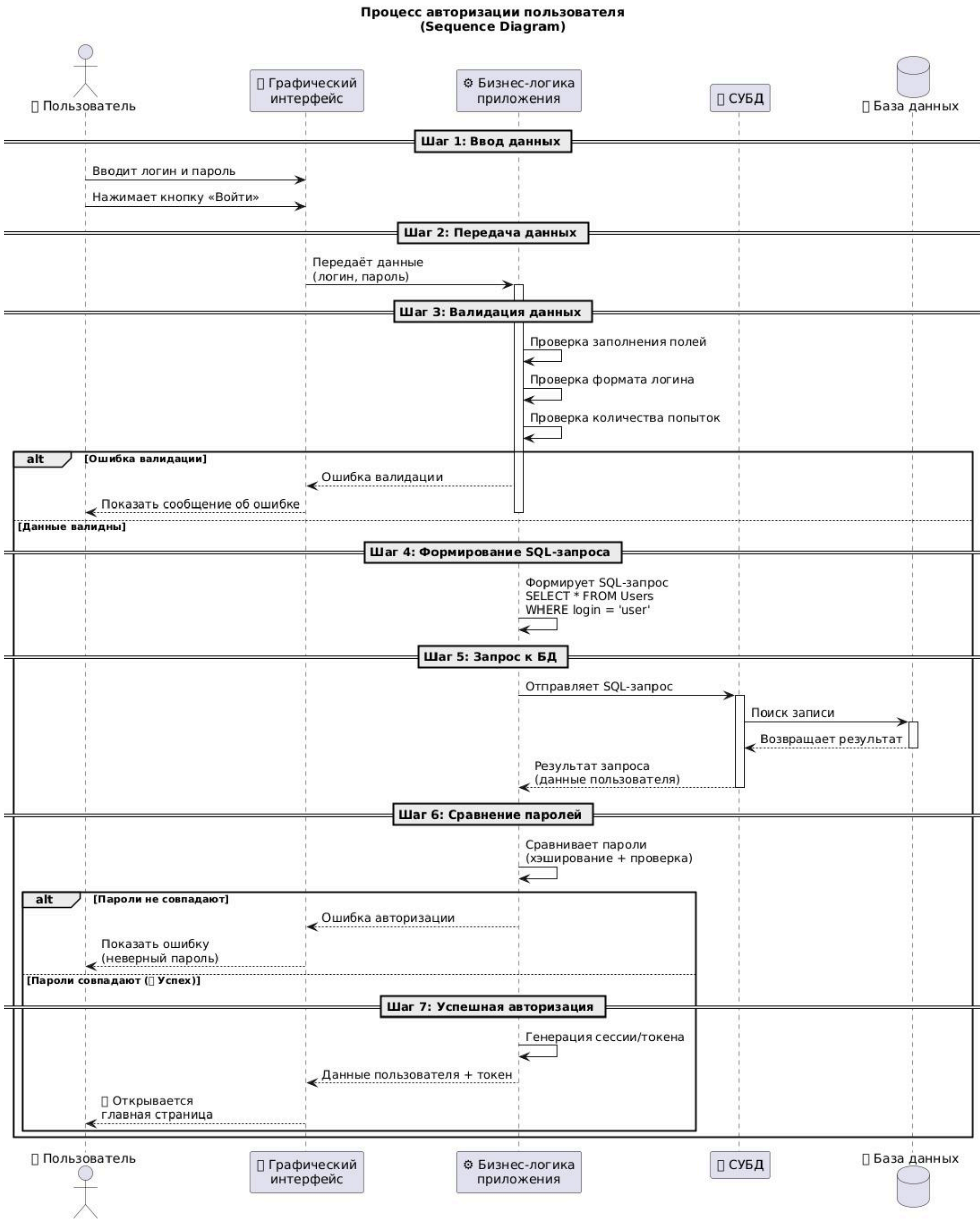
◆ СУБД

- Принимает SQL-запросы
 - Находит данные
 - Контролирует права доступа
 - Проверяет ограничения
 - Возвращает результат
-

◆ База данных

- Хранит информацию
- Сама ничего не делает
- Не отображает окна
- Не проверяет пароли

Пример работы системы (авторизация)



Почему важно понимать архитектуру системы

❌ Частые ошибки начинающих разработчиков

❌ Неправильно

«База данных показывает список товаров»

«SQL открывает страницу сайта»

«MySQL отображает интерфейс»

✅ Правильно

«Интерфейс отображает данные из БД»

«Бизнес-логика формирует страницу»

«СУБД управляет доступом к данным»

Каждый компонент выполняет только свою задачу:

Компонент

Задача

🖥️ Интерфейс

Отображает информацию

⚙️ Бизнес-логика

Принимает решения

💾 СУБД

Управляет доступом к данным

🗄️ База данных

Хранит информацию

Основные требования к современной базе данных

1. Надёжность хранения

Данные не исчезают после перезагрузки или сбоя

2. Целостность данных

Некорректные данные не сохраняются

 Возраст = -5 →  Недопустимо

 Оценка = 8 →  Недопустимо

 Пустая группа →  Недопустимо

3. Минимизация дублирования

Одна информация — одно место хранения

4. Высокая скорость поиска

Миллионы записей — поиск за доли секунды

5. ⚡ **Высокая скорость изменения**

Тысячи заказов в минуту → мгновенные изменения

6. 👥 **Многопользовательская работа**

Сотни и тысячи пользователей одновременно

7. 🔒 **Безопасность**

Разные уровни доступа для разных пользователей

Роль	Доступ
Студент	Только свои данные
Преподаватель	Данные своей группы
Заведующий	Данные отделения
Администратор	Вся система

8. Масштабируемость

Способность расти вместе с системой

 500 товаров → 100 000 товаров

 100 пользователей → 10 000 пользователей

9. Резервное копирование

Восстановление данных после сбоя

10. Отказоустойчивость

Работа даже при отказе компонентов

Используются:

- Резервные серверы
- Репликация
- Кластерные решения

Итог

Современная БД должна обеспечивать:

- ✓ Надёжность • ✓ Целостность • ✓ Скорость • ✓ Безопасность
- ✓ Масштабируемость • ✓ Отказоустойчивость • ✓ Резервирование

Что такое СУБД

Определение

СУБД (Система Управления Базами Данных) — это **сложная программная система**, предназначенная для управления данными, их хранения, обработки, защиты и предоставления доступа к ним.

Простая аналогия

Библиотека

Библиотека

 Книги

 Библиотекари

 Правила выдачи

 Каталог

СУБД

 База данных

 СУБД

 Механизмы управления

 Индексы

Сами книги без библиотеки существуют, но работать с ними становится крайне неудобно

Основные функции СУБД

1. Хранение и организация данных

- Управляет записью на физический носитель
- Разбивка на блоки
- Оптимизация доступа

2. Обработка запросов

- Анализ SQL-запроса
- Выбор оптимального способа выполнения
- Возврат результата

3. Управление транзакциями

Транзакция — группа операций, которые должны быть выполнены полностью или не выполнены вообще

Пример: перевод денег

1. Списание со счёта 
 2. Зачисление на другой счёт 
- ИЛИ**
3. Списание со счёта 
 4. Зачисление на другой счёт 

4. **Контроль целостности данных**

- Нельзя вставить отрицательный возраст
- Нельзя создать запись без обязательных полей
- Нельзя нарушить связи между таблицами

5. **Управление доступом**

Права	Действие
 Чтение	Просмотр данных
 Добавление	Вставка записей
 Изменение	Обновление данных
 Удаление	Удаление записей
 Администрирование	Полное управление

6. **Резервное копирование и восстановление**

Защита данных от потерь

7. **Оптимизация производительности**

- Индексы
- Кэширование
- Оптимизация запросов

Архитектура СУБД (упрощенная)



Из чего состоит СУБД

Как работает СУБД на практике

Пример выполнения запроса

SELECT * FROM Students WHERE age > 18;

Последовательность действий:

1.  СУБД получает SQL-запрос
2.  Проверяет синтаксис
3.  Проверяет права пользователя
4.  Оптимизирует выполнение
5.  Выбирает способ поиска данных
6.  Обращается к физическому хранилищу
7.  Извлекает нужные строки
8.  Возвращает результат приложению

Пользователь видит только результат

Все остальные шаги выполняются внутри СУБД автоматически

СУБД — это программный комплекс, который обеспечивает:

Функция	Описание
 Хранение и организация данных	Физическое размещение на диске
 Обработка запросов	Выполнение SQL-запросов
 Управление транзакциями	Группы операций «всё или ничего»
 Контроль целостности	Защита от некорректных данных
 Управление доступом	Разграничение прав пользователей
 Резервное копирование	Восстановление после сбоев
 Оптимизация	Высокая производительность

СУБД — ключевой компонент любой современной информационной системы

Что такое индекс

Индекс — специальная структура данных, которая ускоряет поиск информации

Пример без индекса (полный перебор)

Студент 1 → Студент 2 → Студент 3 → ... → Студент 1 000 000

 Медленно! Проверяются все записи

Пример с индексом

Индекс (B-Tree)

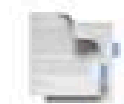


Сразу к нужной записи

 Быстро! Переход к нужной части данных

Аналогия

В жизни



Оглавление в книге

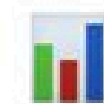


Каталог в библиотеке

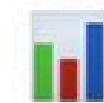


Алфавитный указатель

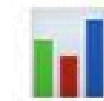
В БД



Индекс по полю



Индекс по автору



Индекс по фамилии

Индексы имеют недостатки:

- Занимают дополнительное место
- Замедляют вставку данных
- Требуют обновления при изменениях

Создаются только там, где это действительно необходимо



Что происходит при выполнении запроса

SELECT * FROM Students WHERE age = 19;

Внутри СУБД:

1. 🔍 Запрос анализируется
2. ✅ Проверяется синтаксис
3. 🗝️ Проверяются права доступа
4. 🎯 Определяется наличие индекса по полю age
5. ⚡ Выбирается оптимальный способ поиска
6. 📁 Загружаются нужные страницы данных
7. 📊 Извлекаются строки
8. 📬 Результат возвращается пользователю

Кэширование данных

СУБД использует кэш — область памяти для часто используемых данных

- ✅ Если страница в кэше → данные из памяти → быстрее
- ❌ Если страницы нет в кэше → чтение с диска → медленнее

Почему существуют разные типы?

Разные модели создавались для решения **различных задач**:

Задача	Подходящий тип
 Иерархические структуры	Иерархические БД
 Сложные связи	Сетевые/Графовые БД
 Структурированные данные	Реляционные БД
 Гибкие документы	Документно-ориентированные БД
 Простые операции	Ключ-значение БД

Классификация

- 1. Иерархические
- 2. Сетевые
- 3. **Реляционные** (основной стандарт)
- 4. Объектно-ориентированные
- 5. Документно-ориентированные (NoSQL)
- 6. Ключ-значение
- 7. Графовые

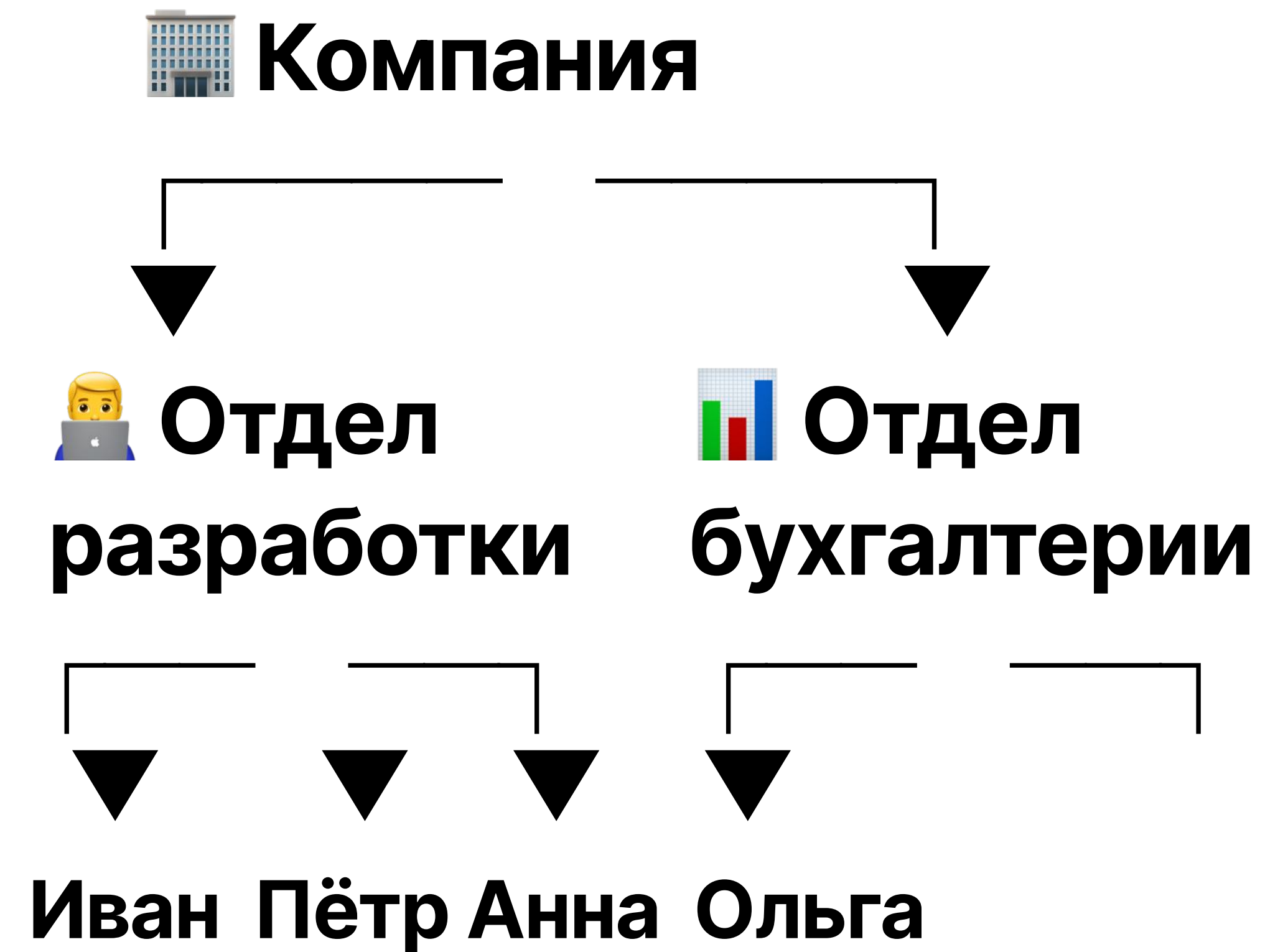
Иерархические базы данных

Особенности

-  Строгая структура «дерево»
-  Быстрый доступ по иерархии
-  Сложность реализации нестандартных связей

Где применяется

-  Файловые системы
-  Организационные структуры
-  Устаревшие корпоративные системы



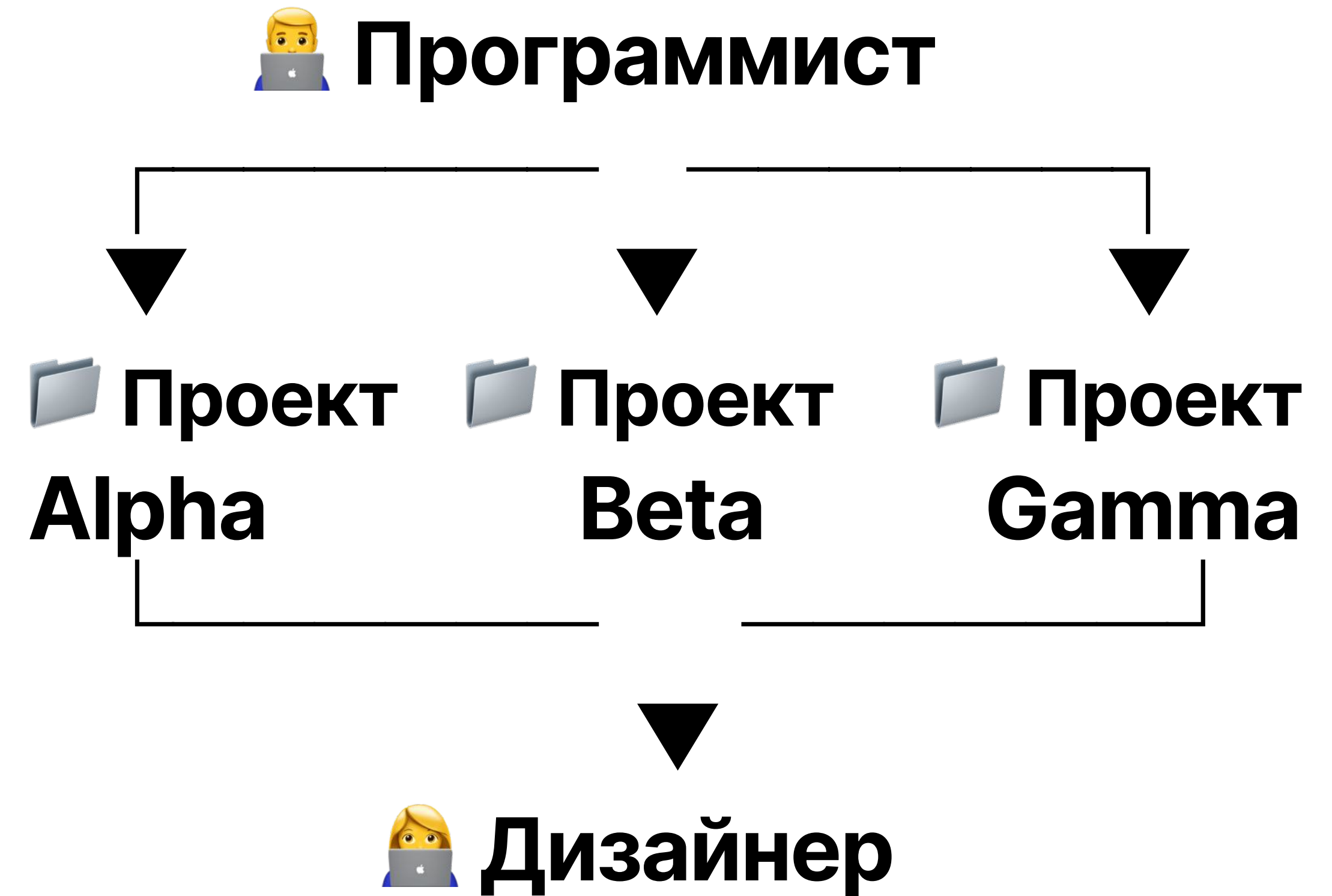
Сетевые базы данных

Особенности

-  Поддержка сложных связей
-  Гибкость структуры
-  Сложность проектирования и сопровождения

Недостаток

-  Сетевые БД трудно масштабировать и поддерживать



Реляционные базы данных

Основные характеристики

-  Данные в таблицах
-  Строки — записи
-  Столбцы — поля
-  Связи через ключи

Где используется

-  Банки
-  Интернет-магазины
-  Государственные системы
-  Образовательные платформы

Таблица: Students

id	name	age
1	Ivan	19
2	Anna	20
3	Peter	18

Объектно-ориентированные базы данных

Особенности

-  Близость к языкам программирования
-  Удобство хранения сложных структур
-  Низкая распространённость

Применение

- Специализированные системы
- Инженерные приложения
- CAD-системы

 **Студент**

Свойства:

- **name: "Ivan"**
- **age: 19**
- **group: "ISP-21"**

Методы:

- + **calculateAvg()**
- + **showInfo()**

Документо-ориентированные БД (NoSQL)

Особенности

-  Гибкая структура
-  Отсутствие строгой схемы
-  Разные поля в разных документах
-  Масштабируемость

Где используется

-  Веб-приложения
-  Мобильные приложения
-  Системы с быстро меняющейся структурой

```
{  
  "name": "Ivan",  
  "age": 19,  
  "group": "ISP-21",  
  "subjects": [  
    "Базы данных",  
    "Web-разработка"  
  ]  
}
```

Ключ-значение (Key-Value)

 Ключ

session_123
token_abc
user_456

 Значение

user_data
authorization_info
Ivan's profile

Особенности

-  Очень высокая скорость доступа
-  Простая структура
-  Отсутствие сложных связей

Где используется

-  Кэширование
-  Сессии пользователей
-  Временные данные
-  Высоконагруженные системы

Ключ-значение (Key-Value)

 Ключ

session_123
token_abc
user_456

 Значение

user_data
authorization_info
Ivan's profile

Особенности

-  Очень высокая скорость доступа
-  Простая структура
-  Отсутствие сложных связей

Где используется

-  Кэширование
-  Сессии пользователей
-  Временные данные
-  Высоконагруженные системы

Графовые БД

Особенности

-  Эффективная работа со сложными связями
-  Быстрый обход связей
-  Удобство анализа сетей

Где используется

-  Социальные сети
 -  Рекомендательные системы
 -  Системы навигации
 -  Анализ связей
-

 Анна

друзья



 Иван

друзья →  Пётр

купил



 iPhone

купил



 MacBook

Сравнение типов БД

Тип	Структура	Скорость	Гибкость	Применение
Иерархические	Дерево	Высокая	Низкая	Файловые системы
Сетевые	Граф	Средняя	Средняя	Устаревшие системы
Реляционные	Таблицы	Высокая	Средняя	Универсальные
Объектно-ООП	Объекты	Средняя	Высокая	Специализированные
Документо-ориент.	JSON	Средняя	Высокая	Веб-приложения
Ключ-значение	Пары	Очень высокая	Низкая	Кэширование
Графовые	Граф	Высокая	Средняя	Соцсети, анализ

